Programação Orientada a Objetos

Aula 04: Programação Orientada a Objetos em Java

classes, objetos e métodos

Prof. Fabricio Santana

2025.1

fabricio.santana@idp.edu.br

www.linkedin.com/in/fabriciofsantana

O que estudamos na última aula?

- Vamos praticar → ***Code Challenge***
  - Em duplas
    - Analisar e corrigir bugs do desafio StringReverse (poo/challenges/01-reverse)
    - Para rodar os testes unitários localmente seguir instruções em poo/challenges
    - Fazer *comit*, push e pull request
    - Primeiro pull request a passar em todos os testes vence o desafio
    - **Tempo: 20 minutos**

Vamos praticar!

- Vamos praticar → ***Code Challenge***
  - Em duplas
    - Analisar e corrigir bugs do desafio StringReverse (poo/challenges/01-reverse)
    - Para rodar os testes unitários localmente seguir instruções em poo/challenges
    - Fazer *comit*, push e pull request
    - Primeiro pull request a passar em todos os testes vence o desafio
    - **Tempo: 20 minutos**

idp

O que vamos estudar hoje?

- Introdução à programação orientada a objetos
- Conceito de classes e objetos
- Anatomia de uma classe (***class***)
- Membros de uma classe: atributos e métodos
- Estrutura do código java: ***module***, ***package***, ***class***

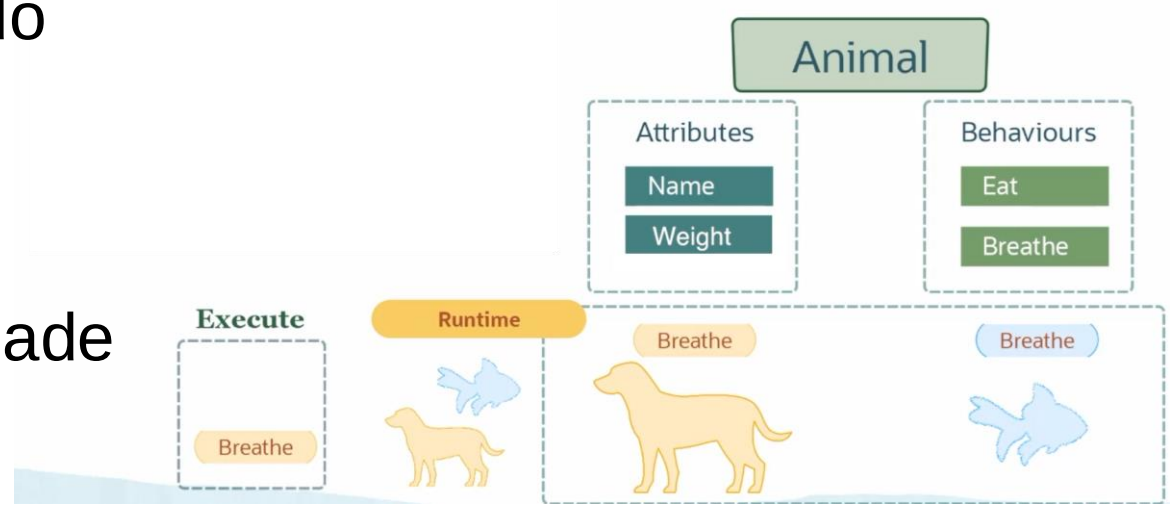
Classes e Objetos

- **Classe**: representa um tipo ou um conceito
  - Possui atributos e comportamento

- **Objeto**: é uma <u>instância</u> (um exemplo específico) de uma classe
  - atributo → variável da instância: mantém os dados
  - Comportamento → método

- Encapsulamento e Herança

- Viabiliza reuso e extensibilidade



Fonte: Oracle University. Oracle Java Fundamentals

Classes e Objetos

- public class <nome da classe> { <corpo da classe> }
  - public: modificador de acesso
  - class: palavra reservada para definição de uma classe (*custom type*)
  - <nome da classe>: identificador da classe, deve ter o mesmo nome do arquivo .java
    - <corpo da classe>
      - <variáveis de instância>: atributos da classe
      - <método>(<tipo> <nome parâmetro>)
        - <corpo do método>
          - <variáveis locais>
          - [return]: para métodos que retornam valor

- Como reutilizar classes java (*built-in types)* ou (*custom types)*
  - import <nome da classe>

Fonte: https://docs.oracle.com/javase/specs/jls/se21/html/jls-8.html

Classes e Objetos

```
public class Charge
{
   private final double rx, ry;
   private final double q;

   public Charge(double x0, double y0, double q0)
   {  rx = x0; ry = y0; q = q0;  }

   public double potentialAt(double x, double y)
   {
      double k = 8.99e09;
      double dx = x - rx;
      double dy = y - ry;
      return k * q / Math.sqrt(dx*dx + dy*dy);
   }

   public String toString()
   {  return q +" at " + "("+ rx + ", " + ry +")";  }

   public static void main(String[] args)
   {
      double x = Double.parseDouble(args[0]);
      double y = Double.parseDouble(args[1]);
      Charge c1 = new Charge(0.51, 0.63, 21.3);
      Charge c2 = new Charge(0.13, 0.94, 81.9);
      double v1 = c1.potentialAt(x, y);
      double v2 = c2.potentialAt(x, y);
      StdOut.printf("%.2e\n", (v1 + v2));
   }

}
```

class name
instance variables
constructor
instance variable names
instance methods
test client
create and initialize object
invoke constructor
object name
invoke method

Fonte:SEGEWICK, Robert; WAYNE, Kevin. Computer Science: An Interdisciplinary Approach. Boston: Addison-Wesley, 2016. ISBN 978-0-13-407642-3.

Classes e Objetos

```java
public class Account {
    private String name;
    private double balance;

    public Account() {
    }

    public Account(String name, double balance) {
        this.name = name;

        if (balance > 0.0) {
            this.balance = balance;
        }
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public void setBalance(double balance) {
        this.balance = balance;
    }

    public double getBalance() {
        return balance;
    }

    public void deposit(double depositAmount) {
        if (depositAmount > 0.0) {
            balance += depositAmount;
        }
    }
}
```

- Atividade em sala, escolher uma dupla
  - Cada dupla: analisar e executar o código das classes
    - Account.java
    - AccountApp.java
    - Student.java
    - StudentApp.java
    - MaximumFinder.java
    - MethodOverload.java
    - Scope.java

- Discussão em sala

Fonte: https://github.com/fabriciosantana/poo/tree/2025.1/lectures/examples/03-custom-types/src
Fonte: https://github.com/fabriciosantana/poo/tree/2025.1/lectures/examples/06-methods/src

Classes e Objetos

```java
public class Account {
    private String name;
    private double balance;

    public Account() {
    }

    public Account(String name, double balance) {
        this.name = name;

        if (balance > 0.0) {
            this.balance = balance;
        }
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public void setBalance(double balance) {
        this.balance = balance;
    }

    public double getBalance() {
        return balance;
    }

    public void deposit(double depositAmount) {
        if (depositAmount > 0.0) {
            balance += depositAmount;
        }
    }
}
```

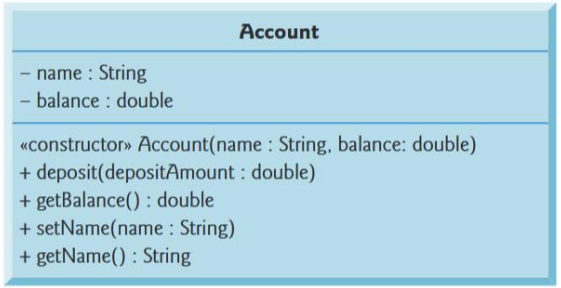


Fig. 3.10 | UML class diagram for Account class of Fig. 3.8.

Fonte: https://github.com/fabriciosantana/poo/tree/2025.1/lectures/examples/03-custom-types/src

Classes e Objetos

```java
public class Account {
    private String name;
    private double balance;

    public Account() {
    }

    public Account(String name, double balance) {
        this.name = name;

        if (balance > 0.0) {
            this.balance = balance;
        }
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public void setBalance(double balance) {
        this.balance = balance;
    }

    public double getBalance() {
        return balance;
    }

    public void deposit(double depositAmount) {
        if (depositAmount > 0.0) {
            balance += depositAmount;
        }
    }
}
```

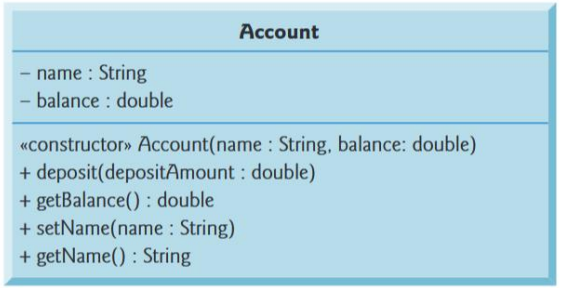


Fig. 3.10 | UML class diagram for Account class of Fig. 3.8.

Fonte: https://github.com/fabriciosantana/poo/tree/2025.1/lectures/examples/03-custom-types/src

Executando um programa

- `javac -d bin src/Account.java src/AccountApp.java`
  - `javac -d bin src/*.java`

- `java -cp bin/ AccountApp.java`

- `jar --create --file bin/AccountApp.jar --main-class AccountApp -C bin/ *.class`

- `java -jar bin/AccountApp.jar`

https://docs.oracle.com/en/java/javase/21/docs/specs/man/index.html

Classes, Objetos e Métodos

- Ajudam a modularizar o software
- Separa responsabilidades em uma unidade auto contida
- Estratégia: divide-and-conquer
  - um software deve ser construído por pequenos e simples pedaços
- Contribui para reutilização
- Evita repetição de código (*copy-and-past*)

Fonte: https://github.com/fabriciosantana/poo/tree/2025.1/lectures/examples/03-custom-types/src

Classes, Objetos e Métodos

- **Scanner**
  - Objeto para receber entrada do usuário
  - try-with-resources

- **new**
  - Palavra reservada para instanciar (criar) um objeto de uma classe

- **construtor**
  - método especial que inicializar um objeto
  - não pode retornar um valor

- ***static method* (*class method*) e static field (variables)**
  - método/atributo da classe que não precisa de um objeto para ser chamado

- **import (package)**
  - viabiliza o reuso

```java
public class AccountApp {

    Run main | Debug main
    public static void main(String[] args) {
        try(Scanner scanner = new Scanner(System.in)){

            System.out.println("Welcome to the Account App!");

            System.out.println(">>> First Account!");

            System.out.println(">>>");

            System.out.print("Enter the account holder's name: ");
            String name = scanner.nextLine();

            System.out.print("Enter the account holder's balance: ");
            double balance = scanner.nextDouble();

            scanner.nextLine();

            Account account = new Account();
            account.setName(name);
            account.setBalance(balance);

            System.out.printf("%s's account balance is $%.2f.\n",
                    account.getName(),
                    account.getBalance());

            System.out.println(">>>");

            System.out.println(">>> Second Account!");

            System.out.println(">>>");

            System.out.print("Enter the account holder's name: ");
            String name2 = scanner.nextLine();

            System.out.print("Enter the account holder's balance: ");
            double balance2 = scanner.nextDouble();

            scanner.nextLine();

            Account account2 = new Account(name2, balance2);

            System.out.printf("%s's account balance is $%.2f.\n",
                    account2.getName(),
                    account2.getBalance());

            System.out.println(">>>");

            System.out.println(">>> Depositing to the First Account!");

            System.out.println(">>>");

            System.out.print("Enter the deposit amount: ");
            double depositAmount = scanner.nextDouble();
            account.deposit(depositAmount);

            System.out.printf("%s's account balance is $%.2f.\n",
                    account.getName(),
                    account.getBalance());
        }
    }
}
```

Fonte: https://github.com/fabriciosantana/poo/tree/2025.1/lectures/examples/03-custom-types/src

Classes, Objetos e Métodos

- Modificadores de acesso
  - `public`
  - `private`
  - `protected`
- Constante
  - `final`
  - `enum`

Classes, Objetos e Métodos

- Há três formas de chamar um método:
  - Usar o nome do método para chamar outro método na mesma classe
    - `double result = maximum(number1, number2, number3);`
  - Usar uma variável que tem referência para um objeto, seguido de um ponto (.) e o nome do método (não estático)
    - `Account account = new Account(name, balance);`
    - `account.deposit(depositAmount);`
  - Usar o nome da classe e um ponto (.) para chamar um método estático

Classes, Objetos e Métodos

- Há três para retornar o controle para quem chamou o método:
  - `void`
  - `return;`
  - `return` *expression*;

Classes, Objetos e Métodos

- **Sobrecarga de método (*method overloading*)**
  - **Métodos com o mesmo nome**, mas com **assinaturas diferentes** (conjunto de parâmetros diferentes, determinados pela quantidade, tipos e ordem dos parâmetros)
    - O tipo de retorno não integra a assinatura
  - Usado para criar vários métodos com o mesmo nome, mas com tipos ou parâmetros diferentes

Classes, Objetos e Métodos

- Escopo
  - parte do código em que se pode acessar uma entidade
    - **parâmetro**: corpo do método em que foi declarado
    - **variável local**: bloco em que foi declarada
    - **método e atributo da classe**: corpo da classe

idp

Estrutura do código em java

- ***Classe***
  - Unidade básica de código
  - Descreve os dados e a lógica

- ***Package***
  - Agrupador de classes (agregador)

- ***Module***
  - Agrupador de pacotes

2ª Atividade

- Implementar os programas
  - Calculadora de frequência cardíaca alvo
  - Registro de saúde digital
  - Palíndromos
  - Sequência de Collatz
  - Mediana de três

- Prazo: 07/03/2025 às 8h15

- **Relembrando**: 2ª leitura para 07/03/2025 até 8h15

INSTITUTO BRASILEIRO DE ENSINO, DESENVOLVIMENTO E PESQUISA

fabricio.santana@idp.edu.br
www.linkedin.com/in/fabriciofsantana